Architecture and System Blueprint: Hyper-Local WhatsApp Commerce Progressive Web Application

Technical Architecture and Feasibility Engineering

Deploying an e-commerce platform using a static Progressive Web Application (PWA) with Google Sheets as the database introduces operational and infrastructural constraints. The system must handle high catalog volumes, maintain low network overhead, bypass cross-origin browser restrictions, and protect administrative credentials without a traditional server runtime.

Scalability Limits and Query Pipelines for Large-Scale Google Sheets

Operating Google Sheets as a relational datastore requires working within platform constraints. A Google Spreadsheet has a maximum ceiling of 10,000,000 cells[1]. A catalog scaling up to 100,000 stock-keeping units (SKUs) across 15 standard product attributes occupies roughly 1,500,000 cells. While this falls within Google's structural capacity[1], client-side web clients encounter performance bottlenecks when processing large spreadsheets directly over mobile connections.

Transporting an uncompressed Comma-Separated Values (CSV) export of 100,000 rows requires transferring 15 MB to 28 MB of raw text across the network. Parsing a payload of this size using JavaScript locks the browser main thread, causing interface freezes on budget mobile hardware common in Indian retail environments.

To maintain sub-second response times without server infrastructure, the data retrieval pipeline uses a tiered query architecture. The primary operational layer, designated as the Active Catalog Tier, serves the merchant's high-velocity inventory (up to 2,500 SKUs). This layer is fetched via Google Sheets "Publish to the Web" as a public CSV endpoint. Ingestion runs off the main thread inside a dedicated Web Worker using PapaParse with chunked streaming. Parsed records are stored locally in the browser using Dexie.js, an IndexedDB abstraction, enabling instant client-side searches and offline access.

For product catalogs extending to 100,000 items, the platform switches from bulk client synchronization to an on-demand Deep Catalog Query Engine. This engine queries a serverless Google Apps Script (GAS) Web App endpoint that exposes Google's native Sheet.createTextFinder(query) API. Because TextFinder executes in Google's cloud infrastructure in under 400 milliseconds, it avoids client-side memory strain by returning only a paginated JSON payload of matching records.

Architectural Parameter	Published CSV Endpoint	Google Apps Script (doGet Endpoint)	Google Sheets REST API v4

Authentication Requirement	None (Publicly Published)	None (Execute as Me, Access: Anyone)	OAuth 2.0 Token or Google API Key
Rate Limits and Quotas	Google Edge Cache; no documented request caps	30 concurrent executions; 6 min execution runtime cap	300 requests per minute per project
Latency Profile	250 ms – 500 ms (Edge Cached)	900 ms – 2,400 ms (V8 Cold Starts)	350 ms – 800 ms
Catalog Suitability	1 to 5,000 active SKUs	Up to 100,000 SKUs via TextFinder	Not viable for zero-backend architecture
Client Processing Strategy	Web Worker stream into IndexedDB	On-demand debounced text search	Direct client calls expose private API keys

The Instagram Media Hotlinking Vulnerability and Resilient Asset Storage

Direct hotlinking of public Instagram image URLs within an e-commerce catalog is fundamentally unstable[2]. Meta implements strict asset protections across its Content Delivery Network (CDN)[2]. All media URLs derived from public Instagram web assets or the Graph API include signed cryptographic parameters, specifically the oe (hex-encoded Unix epoch expiration timestamp) and oh (hash signature) tokens[4].

Within 24 to 72 hours of generation, these signatures expire automatically, returning HTTP 403 Forbidden errors and leaving storefronts with broken image placeholders[2]. Because a static frontend cannot run automated token-refresh routines or scrape fresh CDN hashes without an intermediary server[2], relying on Instagram hotlinking causes critical catalog failures[6].

https://scontent.cdninstagram.com/v/t51.2885-15/..jpg?_nc_cat=1&ccb=1-7&_nc_sid=..&efg=..&_nc_ht=scontent.cdninstagram.com&oh=00_AY..&oe=66F4A123

To maintain a zero-backend architecture while keeping merchant onboarding friction low, the system replaces manual Instagram hotlinking with direct client-side uploads to immutable third-party object storage.

The primary storage workflow uses Cloudinary Unsigned Upload Presets. When managing inventory in the merchant dashboard, the user selects product photos directly from their mobile camera or photo gallery. The client-side application issues a multipart POST request directly to the Cloudinary API using an unsigned upload preset configured for catalog assets. This preset processes the asset, applies auto-format conversions to modern formats like WebP (f_auto,q_auto), resizes the image for mobile displays (w_600), and returns a permanent HTTPS URL.

This permanent URL is written back to the merchant's Google Sheet catalog tab, ensuring images remain available long-term without recurring hosting costs on Cloudinary's free tier (which includes 25 GB of monthly bandwidth and storage).

As a zero-cost backup workflow, the platform supports ImgBB API direct integrations and public Google Drive thumbnail transformations (https://lh3.googleusercontent.com/d/{FILE_ID}), guaranteeing resilient image rendering across all storefronts.

Cross-Origin Protocol Optimization and Google Apps Script Integration

Static web applications hosted on Netlify face Cross-Origin Resource Sharing (CORS) blocks when writing data back to Google Sheets via Google Apps Script (GAS) Web Apps[9]. When a browser application attempts an asynchronous mutation using fetch() with a Content-Type header of application/json, modern browsers flag it as a complex request and dispatch a preflight OPTIONS request to verify server support[9].

Google Apps Script Web Apps do not implement an HTTP OPTIONS handling method[11]. Functions named doOptions(e) are unsupported in the GAS execution engine, and preflight requests fail with an HTTP 405 Method Not Allowed error, terminating the request before reaching the spreadsheet[11].

The application bypasses this limitation by sending mutations as CORS Simple Requests[11]. Simple Requests do not trigger preflight checks, allowing the POST payload to reach the backend directly[11].

To implement this bypass:

1. The client sets the HTTP request method strictly to POST and the Content-Type header to text/plain;charset=utf-8[10].
2. The payload is serialized into a string using JSON.stringify(payload) and placed in the request body[10].
3. The Google Apps Script doPost(e) function reads the payload from e.postData.contents, parses it using JSON.parse(), and writes the updates to the sheet[11].
4. The script wraps its response using ContentService.createTextOutput(JSON.stringify(response)).setMimeType(ContentService.MimeType.JSON), ensuring the browser processes the returned data without CORS errors[10].

```javascript
// Google Apps Script Implementation (Code.gs)
function doPost(e) {
  var lock = LockService.getScriptLock();
  lock.tryLock(10000);
  try {
    var rawData = e.postData.contents;
    var payload = JSON.parse(rawData);
    var ss = SpreadsheetApp.getActiveSpreadsheet();

    if (payload.action === 'UPDATE_CATALOG') {
      var sheet = ss.getSheetByName('catalog');
      var rows = payload.items; // Array of product objects
      // Batch write logic to catalog sheet
      return ContentService.createTextOutput(
        JSON.stringify({ status: 'SUCCESS', message: 'Catalog updated' })
      ).setMimeType(ContentService.MimeType.JSON);
    }

    return ContentService.createTextOutput(
      JSON.stringify({ status: 'ERROR', message: 'Unknown action' })
    ).setMimeType(ContentService.MimeType.JSON);
  } catch (err) {
    return ContentService.createTextOutput(
      JSON.stringify({ status: 'ERROR', message: err.toString() })
    ).setMimeType(ContentService.MimeType.JSON);
  } finally {
    lock.releaseLock();
  }
}
```

Zero-Backend Cryptographic Authentication and Identity Verification

Providing administrative access without third-party identity providers (such as Google OAuth or Firebase) or SMS OTP infrastructure introduces the challenge of verifying merchant identities without exposing passwords in client-side code or browser developer network inspectors. Hardcoding credentials in static source files or sending cleartext passwords over unauthenticated endpoints creates critical security vulnerabilities.

The platform addresses this by implementing a salted cryptographic challenge-response

protocol built on the browser-native Web Crypto API (window.crypto.subtle).
Each merchant's Google Spreadsheet includes a protected tab named _auth_vault that stores three administrative values: an administrative identifier (admin_user), a random 32-byte hexadecimal salt (merchant_salt), and a derived cryptographic hash (password_hash). This hash is pre-computed using the Password-Based Key Derivation Function 2 (PBKDF2) algorithm with 100,000 iterations of SHA-256.
When a merchant initiates an administrative session:

1. The client requests the public salt associated with the store identifier.
2. The user enters their passphrase locally. The browser uses crypto.subtle.deriveBits() with PBKDF2 to compute the matching cryptographic key inside client memory, immediately clearing the plaintext passphrase.
3. The client constructs an authorization token using a Hash-based Message Authentication Code (HMAC-SHA256). The token signs the current store slug alongside a UTC Unix epoch timestamp:

$$\text{AuthToken} = \text{HMAC-SHA256}(\text{DerivedKey}, \text{StoreSlug} \parallel \text{Timestamp})$$

4. The POST request sends only the computed AuthToken and the plaintext Timestamp.
5. The Google Apps Script receives the payload, verifies that the timestamp falls within a five-minute window to prevent replay attacks, and independently calculates the HMAC using the hash stored in _auth_vault.

If both signatures match, the write operation proceeds. This flow ensures that plaintext credentials are never embedded in static production bundles, stored on Netlify CDN edges, or exposed in browser network inspection logs.

Product Requirements Document

Product Scope and Transaction Dynamics

"Vocal For Local" provides independent neighborhood retailers—including grocery stores, pharmacies, bakeries, and regional specialty shops—with an automated digital storefront backed entirely by Google Sheets. The platform operates without server overhead, marketplace aggregation fees, or mandatory payment gateway integrations.
The operational flow prioritizes simplicity. Customers browse the storefront, assemble a cart, enter any special requests, and initiate checkout. The application formats the order into an itemized summary and transfers it directly into the merchant's personal or business WhatsApp conversation via deep link. The merchant confirms product availability, collects payment offline (via cash on delivery, credit ledger, or direct UPI QR code), and coordinates local fulfillment using their existing delivery staff.

Multi-Tenant Storefront and Custom Domain Routing Architecture

The platform runs as a single Progressive Web Application deployment on Netlify, serving multiple independent storefronts through two routing mechanisms:

- **Subdirectory Path Routing (Standard Architecture):** Visiting

https://vocalforlocal.store/s/{store_slug} triggers the client-side router to extract the {store_slug} parameter. The client matches this slug against an internal registry sheet, retrieves the merchant's published Google Sheet ID, and renders their customized storefront.

- **Custom Domain / CNAME Routing (Whitelabel Architecture):** Merchants can link their own branded domain (e.g., https://shreeshivamart.shop) by creating a CNAME record pointing to Netlify's DNS infrastructure. On load, the application inspects window.location.hostname. If the hostname differs from the primary platform domain, the router resolves the domain against the registry sheet, loads the corresponding Google Sheet ID, and renders the merchant's branded catalog without showing platform branding.

The user interface follows a three-stage customer funnel:

- **Platform Root (/):** A merchant-facing marketing portal detailing platform capabilities, featuring a cost-savings calculator, demo storefront sandboxes, and a WhatsApp onboarding channel.
- **Store Home & CMS (/s/:store_slug):** Displays merchant branding, operating hours, physical street address, an embedded Google Maps navigation link, verified business badges, and curated product collections.
- **Product Catalog & Detail Views (/s/:store_slug/p/:sku_id):** Browsing views with two-column product grids, tag-based category filtering, specification tables, and persistent cart modifiers.

Data Schema Specifications

Each merchant operates an isolated Google Spreadsheet containing three primary functional tabs.

Sheet 1: store_config

Maintains global business settings and visual branding configurations.

Column Header	Data Type	Validation Rules / Format	Description
store_slug	String	Alphanumeric, lowercase, hyphenated	Unique route identifier (e.g., gupta-kirana)
store_name	String	Max 60 characters	Display trade name of the merchant
whatsapp_number	String	Country code + digits only (e.g., 919876543210)	Target phone number for WhatsApp orders[16]

instagram_handle	String	Alphanumeric string	Instagram profile handle without @ symbol
logo_url	URL	Valid HTTPS image link	Store avatar image (1:1 aspect ratio)
banner_url	URL	Valid HTTPS image link	Store header banner image (16:9 ratio)
address_text	String	Multi-line text block	Physical address for store pickup and verification
maps_embed_url	URL	Valid Google Maps URL	Location coordinates or Google Maps URL
description	String	Max 500 characters	Merchant summary and operational specialty
currency_symbol	String	Default: ₹	Currency symbol used across catalog calculations
minimum_order	Number	Integer $\geq$	Minimum cart total required for checkout
delivery_fee	Number	Numeric value $\geq$	Flat delivery fee applied at checkout

Sheet 2: catalog

Maintains product inventories, pricing, visual assets, and categorization metadata.

Column Header	**Data Type**	**Validation Rules / Format**	**Description**

sku_id	String	Unique alphanumeric string	Stock Keeping Unit code (e.g., GROC-ATT-001)
name	String	Required string, max 100 characters	Display title of the product
category	String	Taxonomy string	Product category (e.g., Grains & Flours)
mrp	Number	Currency value with two decimals	Maximum Retail Price (strikethrough display)
offer_price	Number	Currency value; $\text{offer_price} \leq \text{mrp}$	Final selling price calculated in cart
images	String	Comma-delimited list of HTTPS URLs	Image links rendered in media carousels
description	String	Plain text or Markdown-lite	Detailed product summary and descriptions
specs_json	JSON String	Valid JSON (e.g., {"Weight":"5kg"})	Key-value pairs displayed on the PDP
tags	String	Comma-delimited strings	Badges (e.g., Bestseller,New Arrival)
rating	Number	Decimal value between 1.0 and 5.0	Average customer review score

review_count	Number	Non-negative integer	Total count of recorded customer reviews
in_stock	Boolean	TRUE or FALSE	Stock flag (disables ordering when false)

Sheet 3: orders_log

Provides an administrative log of submitted orders, populated via Google Apps Script.

Column Header	Data Type	Format	Description
order_id	String	Alphanumeric string	Unique order reference code
timestamp	ISO Date	YYYY-MM-DD HH:mm:ss	Time the order payload was generated
items_summary	String	Serialized text representation	Ordered SKUs, quantities, and line-item totals
total_payable	Number	Decimal currency value	Final order amount including delivery fees
custom_notes	String	Free-form text string	Special requests or unlisted item notes

User Experience and Interface Specifications

The Platform Landing Page functions as a merchant acquisition funnel and technology showcase. The interface includes an animated hero canvas displaying live storefront examples, an interactive calculator showing profit retention compared to aggregator platforms, a live sandbox where merchants can test the WhatsApp ordering flow, and a direct WhatsApp onboarding link to the platform sales team.

The Storefront Interface provides a clean, mobile-optimized shopping experience. The header

displays the store logo, business name, operational status (Open/Closed based on local device time), physical address, and a Google Maps shortcut. Social links for WhatsApp and Instagram are fixed in the storefront footer, giving shoppers direct communication channels. A persistent search bar with dynamic category filter pills allows quick catalog navigation.

The Product Listing Page (PLP) organizes inventory into a responsive two-column grid on mobile viewports (grid-cols-2), expanding to three or four columns on larger screens. Product cards feature multi-image carousels with dot indicators, SKU identifiers, product titles, offer prices, strikethrough MRP values, and dynamic discount percentages (e.g., 15% OFF). Prominent "ADD" action buttons transform into numeric steppers ([-] [1] [+]) upon item selection.

The Product Description Page (PDP) provides detailed product information via a full-width image gallery with touch-swipe navigation, a pricing breakdown displaying total customer savings, and a structured specifications table parsed from the specs_json field. An expandable details accordion displays usage recommendations and storage guidelines. A sticky bottom bar includes a quantity selector and an "Add to Bag" button.

A dedicated Unlisted Items Module addresses ad-hoc requests common in local neighborhood commerce (such as custom butcher cuts, regional brands, or specific produce ripeness). The interface features a prominent text input area within the catalog view and slide-out cart drawer labeled **"Unlisted Items & Special Instructions"**. Placeholder text guides the shopper: *"Need an unlisted item, fresh produce requests, or special delivery instructions? Enter your notes here..."* The entered text is sanitized and included directly in the WhatsApp order summary.

WhatsApp Order Synthesis and Protocol Encoding

When the shopper taps **"Place Order via WhatsApp"**, the application compiles the cart contents, calculates totals, and formats a structured text summary for WhatsApp[16]. WhatsApp click-to-chat links use the official https://wa.me/{phone_number} endpoint format[16]. The destination number must use the international country code without plus signs, dashes, spaces, or leading zeros (for India: 91 followed by the 10-digit mobile number)[16].

The message uses WhatsApp Markdown formatting: asterisks (*) for bold headings, underscores (_) for italicized notes, and hyphens (-) for item lists[20]. Line breaks must be encoded as %0A to maintain layout structure across mobile apps and WhatsApp Web[17]. The full text string is processed using encodeURIComponent() to ensure URL safety[18].

```
JavaScript
// WhatsApp Order Synthesis Logic
export function compileWhatsAppOrder(storeConfig, cartItems, unlistedNotes, customerInfo)
 const cleanPhone = storeConfig.whatsapp_number.replace(/\D/g, '')
 const orderId = 'ORD-' + Math.random().toString(36).substring(2, 8).toUpperCase()
 const dateFormatted = new Date().toLocaleString('en-IN',
   day: 'numeric'
```

```
  month: 'short',
  hour: '2-digit',
  minute: '2-digit'
});

let message = `🛍️ *NEW ORDER: #${orderId}*\n`;
message += `🏪 *Store:* ${storeConfig.store_name}\n`;
message += `📅 *Timestamp:* ${dateFormatted}\n`;
message += `--------------------------------\n\n`;

message += `📦 *ORDERED ITEMS:*\n`;
let subtotal = 0;
let totalSavings = 0;

cartItems.forEach((item, idx) => {
  const itemTotal = item.offer_price * item.quantity;
  const itemSavings = (item.mrp - item.offer_price) * item.quantity;
  subtotal += itemTotal;
  totalSavings += itemSavings;

  message += `${idx + 1}. *${item.name}*\n`;
  message += `   Qty: ${item.quantity} × ₹${item.offer_price} = *₹${itemTotal}*`;
  if (item.mrp > item.offer_price) {
    message += ` _(Saved ₹${itemSavings})_`;
  }
  message += `\n   SKU: \`${item.sku_id}\`\n\n`;
});

message += `--------------------------------\n`;
message += `💵 *Subtotal:* ₹${subtotal}\n`;
if (Number(storeConfig.delivery_fee) > 0) {
  message += `🛵 *Delivery Fee:* ₹${storeConfig.delivery_fee}\n`;
}
const grandTotal = subtotal + Number(storeConfig.delivery_fee || 0);
message += `💰 *TOTAL PAYABLE: ₹${grandTotal}*\n`;
if (totalSavings > 0) {
  message += `🎉 *Total Savings: ₹${totalSavings}*\n`;
}
message += `--------------------------------\n\n`;

if (unlistedNotes && unlistedNotes.trim().length > 0) {
  message += `📝 *UNLISTED ITEMS / SPECIAL INSTRUCTIONS:*\n`;
  message += `${unlistedNotes.trim()}\n\n`;
```

```
if (customerInfo && (customerInfo.name || customerInfo.address)) {
  message += `👤 *DELIVERY DETAILS:*\n`
  if (customerInfo.name) message += `Name: ${customerInfo.name}\n`
  if (customerInfo.address) message += `Address: ${customerInfo.address}\n`
  message += `------------------------------\n\n`
}

message += `_Sent via ${storeConfig.store_name} Online Catalog_`

return `https://wa.me/${cleanPhone}?text=${encodeURIComponent(message)}`
}
```

Sample WhatsApp Message Payload

```
🛍️ *NEW ORDER: #ORD-9J4K2L*
🏪 *Store:* Shree Shivam Groceries
📅 *Timestamp:* 24 Oct, 05:15 PM
--------------------------------

📦 *ORDERED ITEMS:*
1. *Aashirvaad Sharbati Atta 5kg*
   Qty: 1 × ₹235 = *₹235* (Saved ₹25)
   SKU: `GROC-ATT-001`

2. *Amul Salted Butter 500g*
   Qty: 2 × ₹275 = *₹550*
   SKU: `DAI-BUT-002`

--------------------------------
💵 *Subtotal:* ₹785
🛵 *Delivery Fee:* ₹30
💰 *TOTAL PAYABLE: ₹815*
🎉 *Total Savings:* ₹25
--------------------------------

📝 *UNLISTED ITEMS / SPECIAL INSTRUCTIONS:*
```

[redacted]
Please add 500g fresh coriander leaves and 1 packet curd.
Ring bell at Flat 302, landmark: Near Water Tank.

👤 *DELIVERY DETAILS:*
Name: Amit Deshmukh
Address: Flat 302, Green View Apts, Indiranagar

Sent via Shree Shivam Groceries Online Catalog

Merchant Administrative Console

The administrative panel provides a mobile-optimized management interface, allowing store owners to update catalog items without manually editing their Google Sheet.
The dashboard uses client-side PBKDF2 authentication to protect access. The catalog manager allows merchants to create, update, or archive products through simple web forms. Media management connects directly to Cloudinary using unsigned upload presets, saving permanent image URLs to the catalog sheet and eliminating broken image issues.
For bulk inventory updates, the console includes a CSV import tool. The browser parses uploaded spreadsheets using PapaParse, validates required columns, and dispatches batch updates in 500-row chunks to prevent Google Apps Script execution timeouts.

Commercial Domain Strategy and Brand Identity

Registry Economics and Domain Extension Analysis

Choosing an effective Top-Level Domain (TLD) and brand identity for local Indian retail requires balancing commercial trust, regional name recognition, and long-term renewal expenses.
While specialized extensions like .store and .shop clearly indicate an e-commerce storefront, domain registrars like GoDaddy often use promotional first-year pricing paired with higher standard renewal rates[22]. First-year promotional registrations for .shop and .store are accessible[22], but renewals can rise to ₹1,600 to ₹2,500 per year[22].
Country-code TLDs like .in and .co.in offer stable renewal pricing, strong search engine localization across India, and established domestic consumer trust[23]. Combining familiar cultural retail terms (*Dukaan*, *Bazaar*, or *Kirana*) with .in or .shop provides a balanced foundation for brand recognition and local trust.

Domain TLD	GoDaddy Year 1 Promo	Annual Renewal Cost	Cultural Brand Fit in India	Consumer Trust and Perception
.in	₹99 – ₹399	₹749 – ₹949	Universal national trust across retail,	High trust; seen as an established

			technology, and regional services	Indian commercial operation
.co.in	₹49 – ₹199	₹699 – ₹899	Traditional Indian commercial domain standard	High familiarity among business-to-business and retail operations
.store	₹149 – ₹529	₹1,899 – ₹2,499	Direct e-commerce focus; modern presentation[26]	Clear shopping intent, though renewal pricing requires budget planning[24]
.shop	₹122 – ₹299	₹1,699 – ₹2,199	Intuitive shopping term across urban and Tier-2/Tier-3 retail markets	Universal recognition as an e-commerce destination

Brand Identity Formulations for the Indian Retail Ecosystem

The following platform names are designed to resonate with traditional retail merchants and neighborhood shoppers:

- **ApniDukaan (apnidukaan.in / apnidukaan.shop):**
  - *Cultural Fit:* "Apni Dukaan" translates to *"Our Own Shop"* in Hindi. It carries strong emotional appeal, positioning the service as an independent local alternative to large quick-commerce delivery conglomerates.
- **GalliBazaar (gallibazaar.in / gallibazaar.shop):**
  - *Cultural Fit:* "Galli" refers to neighborhood streets and by-lanes. The name emphasizes community-rooted commerce and hyper-local shopping.
- **KiranaLink (kiranalink.in / kiranalink.store):**
  - *Cultural Fit:* Combines the traditional grocery store term (*Kirana*) with modern connectivity, making it straightforward to pitch to family-owned retail shops.
- **VyaparApp (vyaparapp.in / vyaparapp.shop):**
  - *Cultural Fit:* "Vyapar" translates to trade or commerce. The name aligns well with merchant associations, trade networks, and commercial retail districts.
- **OorBazaar (oorbazaar.in / oorkirana.store):**

- *Cultural Fit:* "Oor" or "Ooru" translates to town, village, or locality across South Indian languages (Kannada, Tamil, Telugu), supporting regional brand recognition in southern markets.

Grassroots Go-To-Market and Merchant Acquisition Strategy

Economic Positioning Against Aggregator Ecosystems

Independent neighborhood retailers face margin pressure from quick-commerce and food-delivery aggregator apps, which typically take 15% to 30% in platform commissions. The platform's value proposition focuses directly on commission savings, customer ownership, and zero technical overhead.

Commercial Factor	**Marketplace Aggregators (Blinkit / Zepto / Swiggy)**	**Vocal For Local Platform Model**
Commission Rates	18% to 30% taken from every customer order	Zero commission (100% of order value retained)
Payout Timelines	Delayed weekly or bi-weekly settlement cycles	Immediate payment via merchant's own counter UPI QR
Customer Data Ownership	Aggregator conceals buyer phone numbers and addresses	Merchant builds direct, recurring WhatsApp relationships
Price & Discount Control	Platforms enforce algorithmic discounts and fee matching	Merchant controls selling prices, MRP, and delivery terms
System Complexity	Requires specialized POS software, scanners, and hardware	Managed via a simple Google Sheet or mobile admin console

Direct Merchant Conversion Script and Field Playbook

Field onboarding teams should engage store owners during mid-afternoon operational lulls (typically 2:00 PM to 4:30 PM), avoiding morning and evening rush periods.

The field representative opens with a direct comparison:

> *"Namaste Sharma-ji. We see quick-commerce delivery apps handling orders in our neighborhood while charging stores 20% to 25% commissions. When your regular*

customers order through those platforms, you lose that business.
*We have built **ApniDukaan** to help you run your own digital storefront on your existing WhatsApp number. We set up your entire catalog using a simple Google Sheet—no expensive computer, software subscriptions, or POS equipment required.*
When your neighborhood customers order through your link, the complete item list, quantities, and totals arrive directly in your WhatsApp chat. Payments go straight to your counter's UPI QR code—no middleman, no delayed payment settlements, and zero commissions.
We can launch your online store in ten minutes using your top 50 products today. Let us show you how a test order looks right now on your phone."

Hyper-Local Physical-to-Digital Conversion Channels

Onboarding merchants is only the first step; driving order volume requires converting their existing foot traffic into repeat digital customers using low-cost physical collateral.

- **Countertop Acrylic QR Stands:** Each onboarded shop receives a branded QR code display placed beside their billing counter or UPI payment scanner. The stand features a clear call to action: *"Avoid the evening rush! Scan here to order on WhatsApp for home delivery."*
- **Delivery Bag Packaging Labels:** Merchants receive adhesive stickers affixed to physical carry bags: *"Enjoyed your purchase? Reorder your monthly groceries from home in 1 tap at [StoreName.shop]."*
- **WhatsApp Status Catalog Broadcasting:** Merchants are shown how to share product updates via WhatsApp Status twice weekly (e.g., *"Fresh grains and cooking oils restocked—tap our link to order for free home delivery"*). This re-engages their existing contact list without digital ad spend.
- **Merchant Association Partnerships:** Demonstrating the platform at local trade association meetings (such as the *Vyapar Mandal*) can drive group adoption across entire retail markets, positioning the tool as a collective defense against large delivery conglomerates.

Master Prompt for Full-Stack Implementation

The following prompt provides an end-to-end specification for generating the complete application codebase.

You are an expert Principal Frontend Architect and Serverless Systems Engineer. Build a production-ready, highly optimized Progressive Web App (PWA) for WhatsApp-based local commerce called "Vocal For Local".

ARCHITECTURAL CONSTRAINTS & STACK:

1. Framework: React 18+ (using Vite), Tailwind CSS for styling, Lucide React for icons.
2. Hosting: Completely backendless static frontend deployable to Netlify.
3. Database Layer: Google Sheets exclusively.
   - Read Operations: Fetch published CSV from Google Sheets using PapaParse running within a Web Worker. For deep searches, support an optional Google Apps Script JSON endpoint.

- Write Operations: Send data to a Google Apps Script Web App doPost endpoint.
- CRITICAL CORS BYPASS: GAS does not support OPTIONS preflight requests. All POST requests must be dispatched using fetch(url, { method: 'POST', headers: { 'Content-Type': 'text/plain;charset=utf-8' }, body: JSON.stringify(payload) }). The GAS endpoint parses the incoming text via JSON.parse(e.postData.contents).

4. Image CDN Architecture: Do NOT hotlink Instagram CDN URLs directly, as Meta uses signed time-limited hashes (oe parameter) that expire within 72 hours. Implement direct-to-Cloudinary image uploads using an unsigned upload preset from the browser, saving the permanent Cloudinary HTTPS URL to Google Sheets.
5. Client-Side Persistence: Dexie.js (IndexedDB) for caching products and search indexing; localStorage for the shopping cart and checkout state.

CORE MODULES REQUIRED:

1. Multi-Tenant Storefront Routing

- Determine the active store using either:
  a. Path routing: /s/:storeSlug
  b. Hostname routing: Reading window.location.hostname and resolving it against a store mapping configuration.
- Path / renders the Platform Landing Page ("Vocal For Local"):
  - Hero banner with animations and marketing copy.
  - Interactive demo storefront showcasing the product catalog and WhatsApp checkout.
  - Commission savings calculator comparing aggregator fees (25%) with Vocal For Local (0%).
  - Direct WhatsApp CTA to onboard new merchants.

2. Storefront Interface (/s/:storeSlug)

- Store Header: Displays logo, store name, verified badge, open/closed status, physical address, and a direct Google Maps link.
- Social Links: Floating WhatsApp action button and footer Instagram profile link.
- Search & Filter: Instant search input with debouncing, accompanied by a horizontal-scrolling category tab bar.
- Product Listing Page (PLP):
  - 2-column responsive mobile grid (grid-cols-2).
  - Product Card: Image carousel (with swipe support for multiple images), SKU ID badge, product name, offer price, strikethrough MRP, percentage discount badge, and tag badges (e.g., "Bestseller").
  - Cart Stepper: "ADD" button transforms into an active quantity counter ([-] [qty] [+]).
- Product Description Page (PDP):
  - Accessible via modal or route /s/:storeSlug/p/:skuId.
  - Full-width image gallery carousel.
  - Key-value specifications table parsed from specs_json (e.g., Weight, Shelf Life, Ingredients).
  - Expandable accordion for product descriptions and usage notes.
- Custom Order / Unlisted Items Module:

- Text area input: *"Can't find an item? Add your custom grocery list or delivery instructions here..."*
- The entered text is appended directly to the final checkout payload.

3. Cart & WhatsApp Checkout System

- Slide-out Cart Drawer: Displays item list, quantities, price calculations, and subtotal.
- Delivery Fee logic: Adds delivery surcharge if defined in store_config.
- "Place Order via WhatsApp" CTA:
  - Formats an itemized order message using WhatsApp Markdown (*bold*, _italic_, line breaks).
  - Uses the official deep link schema: https://wa.me/{clean_phone_number}?text={encoded_message}.
  - Ensures clean phone formatting (digits only, country code included, no + or spaces).
  - Encodes the full payload using encodeURIComponent().

4. Merchant Administration Panel (/s/:storeSlug/admin)

- Zero-Backend Authentication:
  - Validates login credentials against a salted PBKDF2 hash stored in the private _auth_vault sheet tab.
  - Derives the hash client-side using crypto.subtle before verifying.
  - Plain-text passwords must never be stored in code or sent across the network.
- Catalog Management:
  - Form to add and edit products (SKU, Name, Category, MRP, Offer Price, Tags, Specs JSON).
  - Image upload input with direct-to-Cloudinary upload, saving the returned HTTPS link.
  - CSV bulk import tool: Parses CSV files client-side using PapaParse and updates the sheet.

CODE DELIVERABLES:

Generate fully functional, clean, and modular code:

1. src/services/sheetService.js: Data fetching, PapaParse Web Worker integration, and GAS communication.
2. src/services/authService.js: Web Crypto API implementation for salted PBKDF2 hash verification.
3. src/services/whatsappService.js: Order text compiler and URL encoder.
4. src/components/ProductCard.jsx: 2-column PLP card with multi-image carousel and quantity steppers.
5. src/components/CartDrawer.jsx: Cart sidebar with the unlisted items input and checkout button.
6. src/pages/Storefront.jsx: Complete storefront with search, category filtering, and product grids.
7. src/pages/AdminDashboard.jsx: Merchant admin panel with Cloudinary uploads and product forms.
8. google-apps-script/Code.gs: Complete Google Apps Script implementation containing doGet, doPost, Sheet.createTextFinder, and CORS header responses.

Ensure all edge cases are handled, including offline PWA caching, broken image fallbacks, and mobile layout optimizations.

Technical Synthesis and Implementation Roadmap

This architecture provides a scalable e-commerce model designed for neighborhood retail. Using Google Sheets as the database eliminates ongoing server costs and gives merchants a management tool they already know how to use.

Addressing core technical constraints early ensures reliable day-to-day operation: replacing expiring Instagram links with Cloudinary uploads protects catalog images[2], using plain-text POST payloads avoids Google Apps Script CORS issues[10], and client-side cryptographic hashing secures admin access without extra authentication services.

For market rollout, combining cost-effective .in or .shop domain naming with on-the-ground merchant onboarding provides a practical path to scaling[22]. By offering zero platform commissions, direct customer relationships, and immediate UPI settlements, the platform provides neighborhood retailers with a viable, self-managed alternative to large delivery aggregators.

Works cited

1. Limits on data size - AppSheet Help, https://support.google.com/appsheet/answer/10104789?hl=en
2. How to deal with expiring images of TikTok and Instagram - Iframely, https://iframely.com/help/015914-how-to-deal-with-expiring-images-of-tik-tok-and-instagram
3. Image URL "displayURL" expiring · Issues · Instagram Scraper - Apify, https://apify.com/apify/instagram-scraper/issues/image-url-displayurl-7jrNvg29pclZoVarP
4. Check if Instagram Photo Access URL has Expired - Stack Overflow, https://stackoverflow.com/questions/77198797/check-if-instagram-photo-access-url-has-expired
5. CDN images expire · Issues · Instagram Post Scraper - Apify, https://apify.com/apify/instagram-post-scraper/issues/cdn-images-expire-kJ1JJ7t5CmS5BkTrT
6. Instagram Feed – “URL signature expired” issue causing black, https://wordpress.org/support/topic/instagram-feed-url-signature-expired-issue-causing-black-thumbnails/
7. Resolving Instagram Profile Picture Bad URL Hash Issue - Medium, https://medium.com/@python-javascript-php-html-css/resolving-instagram-profile-picture-bad-url-hash-issue-395ebc420270
8. My client lost 3 weeks of Instagram posts because of an expired, https://www.reddit.com/r/zapier/comments/1s7skj8/my_client_lost_3_weeks_of_instagram_posts_because/
9. Fixing CORS Errors in Google Apps Script | Lambda IITH, https://iith.dev/blog/app-script-cors/

10. Google App Script Web App GET and POST request blocked by, https://stackoverflow.com/questions/56502086/google-app-script-web-app-get-and-post-request-blocked-by-cors-policy
11. Google Apps Script Web App Not Handling CORS Preflight ... - Reddit, https://www.reddit.com/r/GoogleAppsScript/comments/1mhps0o/google_apps_script_web_app_not_handling_cors/
12. “Failed to Fetch” to “Registration Success” | by Arjun M - Medium, https://medium.com/@genzarjun/failed-to-fetch-to-registration-success-10bf82b73a99
13. Struggling with CORS in Google Apps Script? Here's the Fix - Medium, https://diyavijay.medium.com/struggling-with-cors-in-google-apps-script-heres-the-fix-e3eec09f07dd
14. No 'Access-Control-Allow-Origin' header is present #154 - GitHub, https://github.com/dwyl/learn-to-send-email-via-google-script-html-no-server/issues/154
15. How do i allow a CORS requests in my google script? - Stack Overflow, https://stackoverflow.com/questions/53433938/how-do-i-allow-a-cors-requests-in-my-google-script
16. How to Create WhatsApp Link: Complete Guide, https://anantya.ai/blog/how-to-create-whatsapp-link-complete-guide/
17. Free wa.me Link Generator: WhatsApp Click-to-Chat ... - NagMeLater, https://nagmelater.com/tools/whatsapp-link-generator.html
18. WhatsApp Link Generator - wa.me Click-to-Chat and Share Links, https://urleditor.online/guides/whatsapp-share-link
19. WhatsApp Link Format (wa.me): every URL parameter, with examples, https://wha.tools/whatsapp-link-format
20. How to Send Clickable Links on WhatsApp (App and API) - Chat Mitra, https://chatmitra.com/blog/send-links-on-whatsapp/
21. Create line break in WhatsApp message - Stack Overflow, https://stackoverflow.com/questions/27780542/create-line-break-in-whatsapp-message
22. How To Buy Domain Name From GoDaddy In 2026 - Forbes, https://www.forbes.com/advisor/in/business/software/how-to-buy-domain-from-godaddy/
23. Comparing the best domain registrars of 2026 in India - GoDaddy, https://www.godaddy.com/resources/in/skills/the-best-domain-registrars-in-india
24. How much does a domain name cost? Find out! - GoDaddy, https://www.godaddy.com/resources/skills/how-much-domain-name-cost
25. Domain Name Search - Find & Register an Available ... - GoDaddy, https://www.godaddy.com/en-in/domains
26. .store Domain Names | Register Your .store Domain - GoDaddy, https://www.godaddy.com/en-in/tlds/store-domain